Dev Back

M1 - DFS

Sommaire



Structure API

Resource & Request



Middleware

Middleware de sécurisation

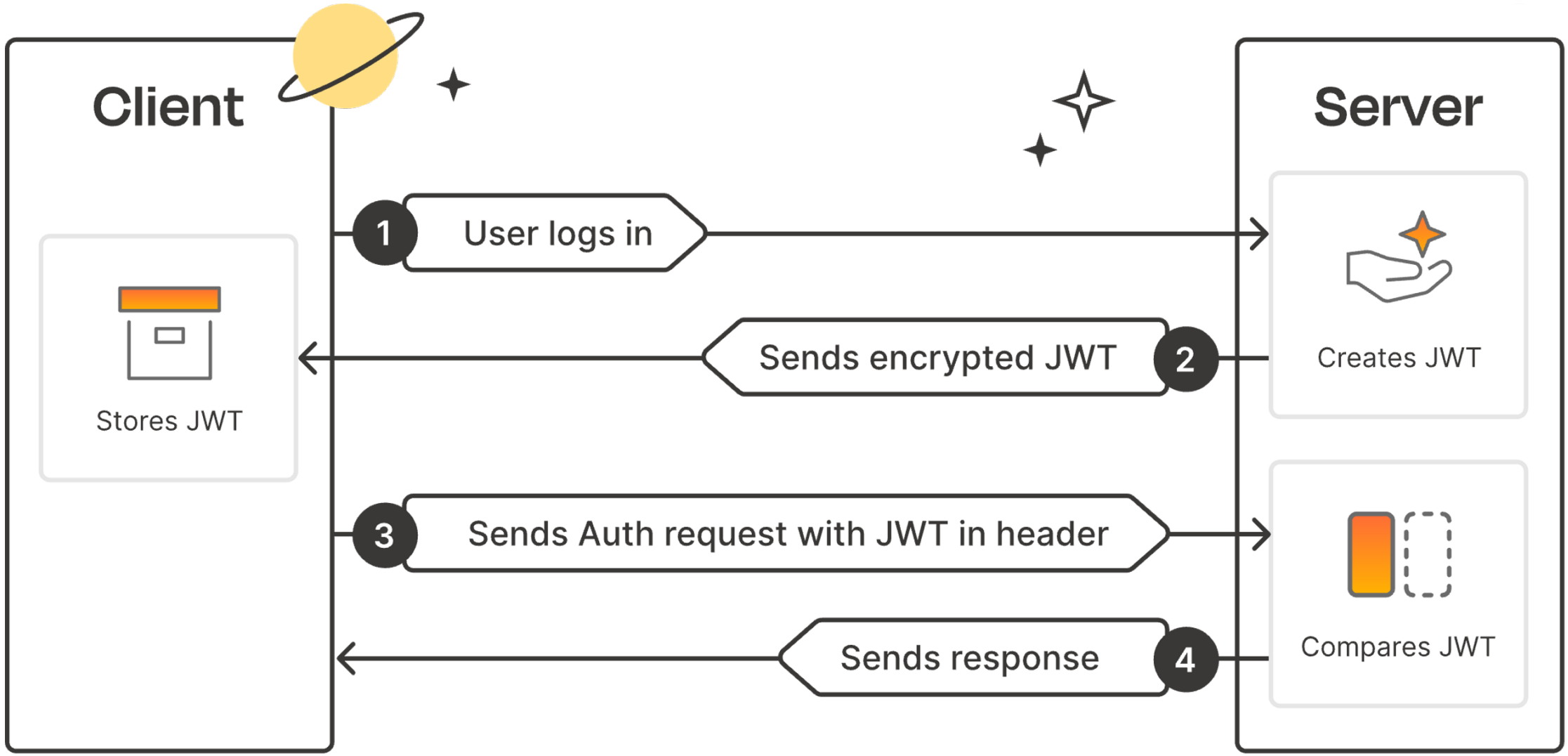
- Throttle
- API Key
- RBAC



Serialization

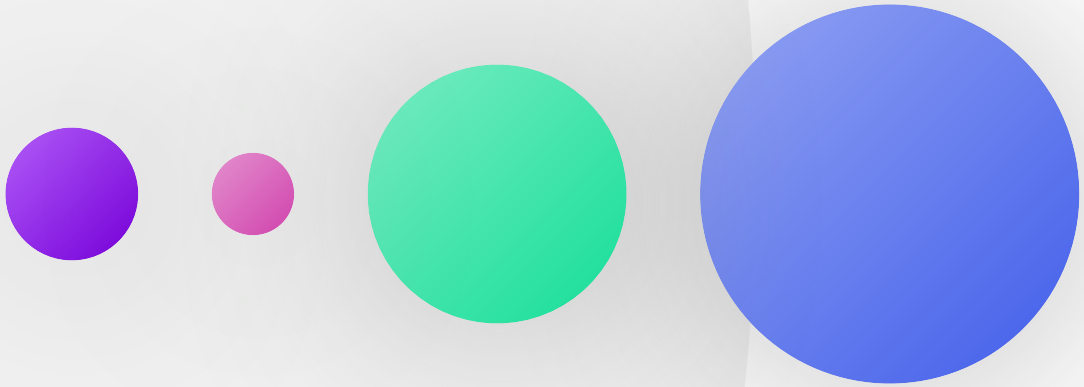
Créer son propre JSON de retour structuré

JWT


Client
Server
1
User logs in
Stores JWT
Sends encrypted JWT
2
Creates JWT
3
Sends Auth request with JWT in header
Compares JWT
Sends response
4

Structure API

Avancée



Structure avancée API

Resources : Permettent de transformer des modèles et collections en JSON structuré

Form Requests : Gèrent la validation des données entrantes et les autorisations

Resources

Une classe qui transforme les modèles ou collections en format JSON structuré

```
php artisan make:resource UserResource
```

- Transformation cohérente des données
- Contrôle précis sur les attributs exposés
- Inclusion conditionnelle de relations
- Formatage personnalisé des attributs
- Séparation claire entre la logique métier et la présentation des données

Resources

```php
<?php

namespace App\Http\Resources;

use Illuminate\Http\Resources\Json\JsonResource;

class UserResource extends JsonResource
{
    /**
     * Transform the resource into an array.
     *
     * @param  \Illuminate\Http\Request  $request
     * @return array
     */
    public function toArray($request)
    {
        return [
            'id' => $this->id,
            'name' => $this->name,
            'email' => $this->email,
            'created_at' => $this->created_at->format('d/m/Y')
            'updated_at' => $this->updated_at->format('d/m/Y')
        ];
    }
}
```

Resources

```
UserResource::make(Auth::user())
```

On fait appelle à la Resource (attention à bien l'importer)

Pour créer un seul item on utilise ::make() avec en argument notre user ou dans ce cas présent, l'utilisateur connecté.

Pour une liste on utilise ::collection() avec en argument une liste de users

Resources

Ajouter de la
data supplémentaire

```php
/**
 * Get additional data that should be returned with the resour
 *
 * @param  \Illuminate\Http\Request  $request
 * @return array
 */
public function with($request)
{
    return [
        'meta' => [
            'version' => '1.0.0',
            'api_status' => 'stable',
        ],
    ];
}
```

Resources


```
"data": {
    "id": 3,
    "email": "super_admin@example.com",
    "role": "super_admin",
    "created_at": "2025-02-28T08:55:50.000000Z",
    "updated_at": "2025-02-28T08:55:50.000000Z"
},
"meta": {
    "version": "1.0.0",
    "api_status": "stable"
}
```

Resources

Ajouter des relations ou autres données complémentaire dans le coeur data : {}

```php
return [
    'id' => $this->id,
    'name' => $this->name,
    'email' => $this->email,
    // Inclusion conditionnelle
    'secret' => $this->when($request->user()->isAdmin(), 'secret-value'),
    // Relations
    'posts' => PostResource::collection($this->whenLoaded('posts')),
];
```

Requests

```php
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class StoreUserRequest extends FormRequest
{
    /**
     * Determine if the user is authorized to make this request.
     *
     * @return bool
     */
    public function authorize()
    {
        return true; // ou logique d'autorisation personnalisée
    }

    /**
     * Get the validation rules that apply to the request.
     *
     * @return array
     */
    public function rules()
    {
        return [
            'name' => 'required|string|max:255',
            'email' => 'required|string|email|max:255|unique:users',
            'password' => 'required|string|min:8|confirmed',
        ];
    }
```

Requests

Exemple personnel

```php
/**
 * Get the validation rules that apply to the request.
 */
public function rules(): array
{
    return [
        'email' => ['required', 'email', 'max:255', 'unique:users,email'],
        'role' => ['required', 'string', 'in:' . implode(',', RoleEnum::valuesAsArray())],
        'password' => ['required', 'string', 'max:255', 'confirmed', Password::min(8)
            ->letters()
            ->mixedCase()
            ->numbers()
            ->symbols()],
    ];
}
```

Requests

```php
/**
 * Get custom messages for validator errors.
 *
 * @return array
 */
public function messages()
{
    return [
        'name.required' => 'Le nom est obligatoire',
        'email.required' => 'L\'email est obligatoire',
        'email.email' => 'L\'email doit être une adresse valide',
        'password.required' => 'Le mot de passe est obligatoire',
        'password.min' => 'Le mot de passe doit contenir au moins 8 caractères',
    ];
}
```

Requests

Personnalisation des messages

```php
/**
 * Get custom messages for validator errors.
 *
 * @return array
 */
public function messages()
{
    return [
        'name.required' => 'Le nom est obligatoire',
        'email.unique' => 'Cette adresse email est déjà utilisée',
    ];
}
```

Requests

Exemple d’utilisation

```php
public function store(StoreUserRequest $request)
{
    // La validation est déjà effectuée
    $user = User::create($request->validated());

    return new UserResource($user);
}
```

Conclusions

Les Resources et les Requests sont des outils puissants de Laravel qui permettent de :

- **Améliorer la structure de votre code** en séparant les responsabilités
- **Renforcer la sécurité** grâce à la validation et aux autorisations
- **Standardiser les réponses API** avec un formatage cohérent
- **Réduire la duplication de code** en centralisant la logique

TP : Prise en main

Sur vos API :

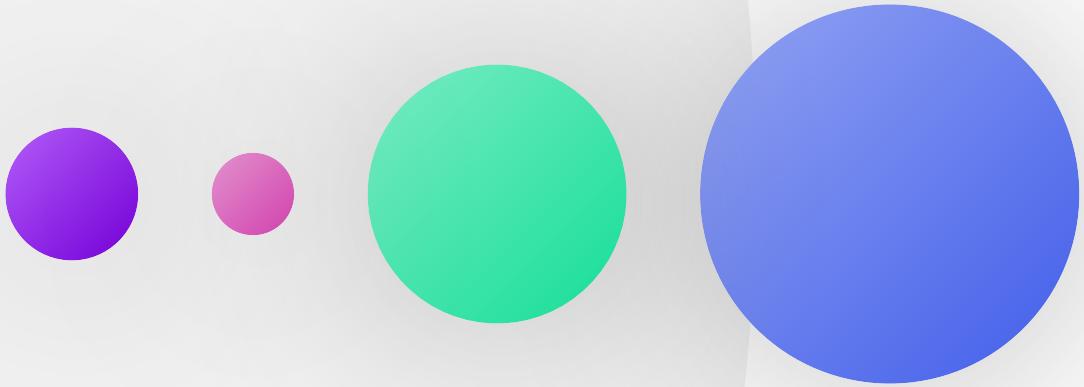
Créer des Resources pour vos Models et utiliser les dans vos réponses JSON

Créer une Request pour vos Models a intégrer dans vos méthode Store/Create en enlevant le validator



Middleware

Sécurisation API



Middlewares

Comment ça marche

- Un middleware est un filtre qui s'exécute avant ou après une requête HTTP
- Il permet de vérifier des conditions avant d'accéder à une route
- La méthode `handle()` est le point d'entrée du middleware
- `$next($request)` permet de continuer vers la prochaine étape
- Les middlewares peuvent être globaux ou spécifiques à certaines routes

Middlewares

Bonnes pratiques

- Toujours utiliser des codes HTTP appropriés (401 pour non autorisé)
- Valider les entrées de manière sécurisée
- Utiliser des messages d'erreur clairs
- Implémenter une gestion d'erreurs robuste
- Documenter le middleware pour les autres développeurs

Middlewares

Throttle (natif) : Permettent de transformer des modèles et collections en JSON structuré

Accès par clé API (perso) : Gèrent la validation d'une clé API en entête des requêtes pour l'accès aux routes

Gestion rôles (perso) : Gèrent l'accès à la route selon le rôle de l'utilisateur qui l'utilise

Throttle

Le middleware `throttle` est un outil puissant pour limiter le nombre de requêtes qu'un client peut effectuer sur votre API ou site web dans un intervalle de temps donné.

C'est essentiel pour :

- Prévenir les attaques par force brute
- Protéger contre les abus d’API
- Éviter la surcharge de votre serveur
- Assurer une répartition équitable des ressources

```php
// Protection contre les attaques par force brute
Route::middleware('throttle:5,1')->group(function () {
```

Middleware ApiKey

Création d'un middleware qui viendra inspecter l'entête de la requête API et interrogera la base de données pour savoir si la clé correspondante existe en base et autorisera la requête ou non

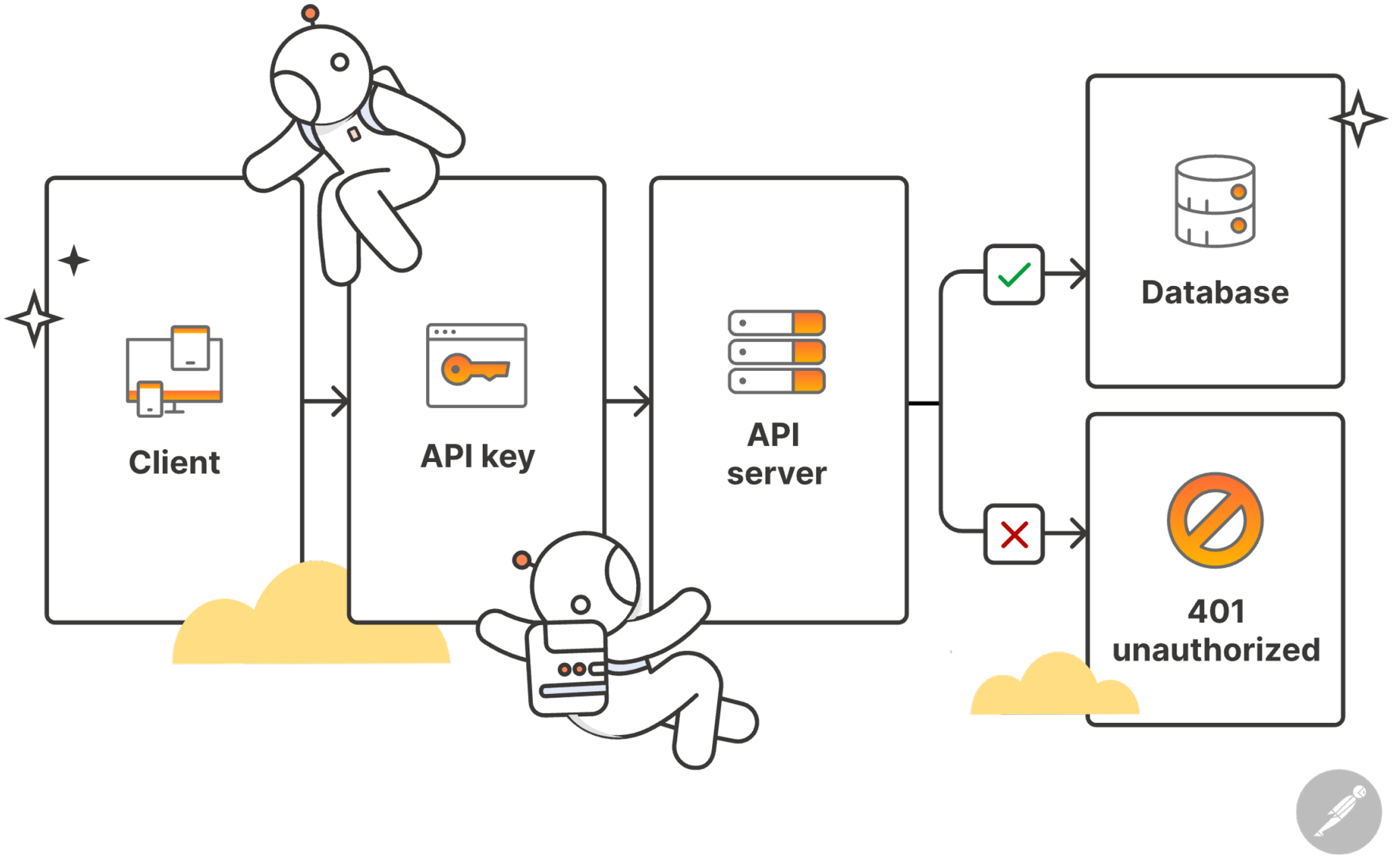
```
php artisan make:middleware CheckApiKey
```

Middleware ApiKey

Le middleware CheckApiKey est un exemple de middleware d'authentification qui :

- Vérifie la présence d'une clé API dans les en-têtes de la requête (Exemple : API-KEY)
- Valide cette clé dans la base de données
- Autorise ou refuse l'accès à la route

API Key



Middleware ApiKey

```php
public function handle(Request $request, Closure $next): Response
{
    // Récupère la clé API depuis les en-têtes de la requête
    $apiKey = $request->header('API-KEY');

    // Vérifie si la clé existe et est valide dans la base de données
    if (!$apiKey || !ApiKey::where('key', $apiKey)->exists()) {
        // Retourne une erreur 401 si la clé est invalide
        return response()->json(['error' => 'Unauthorized'], 401);
    }

    // Continue vers la prochaine étape si la clé est valide
    return $next($request);
}
```

Middleware ApiKey

Attention, bien mettre l'alias de votre middleware dans le fichier Kernel.php dans **app/http** pour l'utiliser dans vos fichiers routes.php

```php
protected $middlewareAliases = [
    'auth' => \App\Http\Middleware\Authenticate::class,
    'auth.basic' => \Illuminate\Auth\Middleware\AuthenticateWithBasicAuth::class,
    'auth.session' => \Illuminate\Session\Middleware\AuthenticateSession::class,
    'cache.headers' => \Illuminate\Http\Middleware\SetCacheHeaders::class,
    'can' => \Illuminate\Auth\Middleware\Authorize::class,
    'guest' => \App\Http\Middleware\RedirectIfAuthenticated::class,
    'password.confirm' => \Illuminate\Auth\Middleware\RequirePassword::class,
    'precognitive' => \Illuminate\Foundation\Http\Middleware\HandlePrecognitiveRequests::class,
    'signed' => \App\Http\Middleware\ValidateSignature::class,
    'throttle' => \Illuminate\Routing\Middleware\ThrottleRequests::class,
    'verified' => \Illuminate\Auth\Middleware\EnsureEmailIsVerified::class,
    'api.key' => \App\Http\Middleware\CheckApiKey::class,
    'role' => \App\Http\Middleware\CheckRole::class,
];
```

TP : Prise en main

Sur vos API :

Créer une entité pour stocker vos clé API, insérer des clés dedans manuellement puis faire le middleware de contrôle de clé API.



Middleware CheckRole

Création d’un middleware qui viendra inspecter le rôle de l’utilisateur afin de valider ou non l’accès à la route

```
php artisan make:middleware CheckRole
```

Middleware CheckRole

Avec le code si dessous expliquez moi le fonctionnement.
Contexte : Dans mon entité User je possède une énum qui contient des rôles.

```php
enum RoleEnum: string
{
    case ADMIN       = 'admin';
    case SUPER_ADMIN = 'super_admin';
```

```php
protected $casts = [
    'email_verified_at' => 'datetime',
    'password' => 'hashed',
    'role' => RoleEnum::class
```

```php
Route::patch('deactivate/{id}', MediaController::class . '@deactivate')->middleware(['role:admin,super_admin']);
Route::delete('delete/{id}', MediaController::class . '@delete')->middleware(['role:admin,super_admin']);
```

```php
public function handle(Request $request, Closure $next, ...$roles): Response
{
    if (!$request->user() || !in_array($request->user()->role->value, $roles)) {
        return JsonApiResponse::make()->withErrors('Forbidden Access')->send(403);
    }

    return $next($request);
}
```

Utilisation des middlewares

```php
Route::patch('deactivate/{id}', MediaController::class . '@deactivate')->middleware(['role:admin,super_admin']);
Route::delete('delete/{id}', MediaController::class . '@delete')->middleware(['role:admin,super_admin']);


Route::group(['middleware' => ['api.key']], function () {

    Route::prefix('wp-media')->group(function () { //! like to change to ext-media instead for wp-media
        Route::post('create-with-tpl', MediaController::class . '@createTpl');
        Route::post('create-without-tpl', MediaController::class . '@createNoTpl');
        Route::get('get', MediaController::class . '@get');

        //Route for increment nb_opened on the onStartMethod react player
        Route::patch('incr-counter/{id}', MediaInformationController::class . '@incrCounter');
    });
});
```

TP : Prise en main

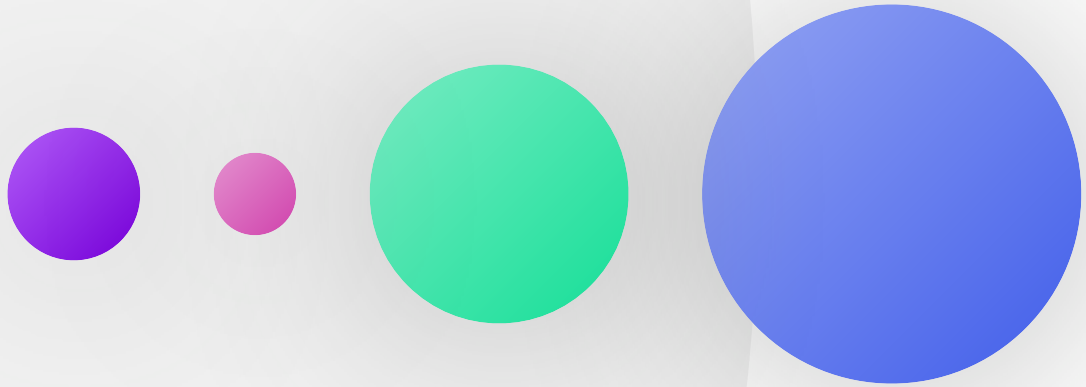
Sur vos API :

Créer un middleware de gestion de contrôle d'accès selon le rôle de l'utilisateur qui correspond à votre structure API (pas de copié collé de mon exemple car j'ai une entité User particulière)



Serialization

JSON



Serialization personnalisée

La standardisation des réponses API est une bonne pratique qui permet de :

- Maintenir une cohérence dans toutes vos réponses
- Faciliter le traitement côté client
- Améliorer la maintenabilité de votre code
- Centraliser la logique de formatage

Serialization personnalisée

Créer une classe permettant de retourner un JSON de ce type.
La classe implémentera l'interface `Responsable` de Laravel, ce qui permet de la retourner directement depuis un contrôleur.

```
{
    "success": true,
    "message": "Utilisateurs récupérés avec succès",
    "data": {
        // Vos données ici
    }
}
```

Serialization personnalisée

Exemple d’utilisation

```php
//en cas de succès
return ApiResponse::success(
    'User fetched successfully', // message d'information
    UserResource::make(Auth::user()), // data
    200 // code de statut 200 par défaut
);
```


```json
{
    "success": true,
    "message": "User fetched successfully",
    "data": {
        "id": 3,
        "email": "super_admin@example.com",
        "role": "super_admin",
        "created_at": "2025-02-28T08:55:50.000000Z",
        "updated_at": "2025-02-28T08:55:50.000000Z"
    }
}
```

Serialization personnalisée

Exemple d’utilisation

```php
//en cas d'erreur
return ApiResponse::error(
    'User not found', // message d'information
    null, // data
    404 // code de statut 404 par défaut
);
```

```json
{
    "success": false,
    "message": "User not found",
    "data": null
}
```

Serialization personnalisée

Pour surcharger les exceptions natives (exemple validation exception pour les requests)
Dans bootstrap/app.php

```
withExceptions
$exceptions->rendarable(function (ValidationException $e)
{       // votre format de réponse
    })
```

TP : Prise en main

Sur vos API :

Créer un fichier ApiResponse (ou autre nom cohérent) et l'utiliser dans vos réponses afin de structurer l'information.

